SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Design Task

Code of Conduct:

I ATHIN SIVA.S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Hybrid AI Recommendation Engine Design

Problem Statement: Design a hybrid AI-based recommendation system that combines content-based filtering with collaborative filtering. The system must use PySpark RDDs to process large-scale user-item data and intelligent agents to adapt recommendations according to real-time user behaviour and feedback.

1.1 Abstract

The Hybrid AI Recommendation Engine is a distributed recommendation platform designed to provide accurate and personalized recommendations for large numbers of users. It combines two major approaches: content-based filtering, which recommends items similar to what a user has previously liked, and collaborative filtering, which identifies users with similar preferences and recommends items that those users prefer. PySpark RDDs are used to distribute the processing of large user profiles, item descriptions, ratings and interaction histories. Intelligent recommendation agents continuously observe user actions such as clicks, searches, ratings, skips and purchases. These agents update user preference profiles and modify recommendation weights in real time. The hybrid architecture improves recommendation quality, reduces the cold-start problem and provides scalable personalization.

1.2 Objectives

Process millions of user-item interactions in a distributed environment.

Combine content similarity and collaborative preference information.

Provide personalized recommendations for different users and contexts.

Use intelligent agents to learn from real-time feedback.

Improve recommendation relevance while maintaining scalable performance.

Support continuous updating without rebuilding the entire system from scratch.

1.3 Proposed Architecture

Component	Description
Data Sources	User profiles, product/movie/course metadata, ratings, clicks, searches, purchases, reviews and browsing history.
PySpark RDD Layer	Stores distributed records and performs map, filter, flatMap, groupByKey, reduceByKey, join and aggregate operations.
Content-Based Module	Converts item descriptions and user preferences into feature representations and calculates similarity.
Collaborative Filtering Module	Finds similar users/items from historical interaction and rating patterns.
Hybrid Scoring Module	Combines content and collaborative scores using weighted ranking.
Intelligent Agent Layer	Observes feedback, updates user preference states and adjusts recommendation behaviour.

Recommendation Interface	**Displays ranked items and collects new interaction data.**

1.4 PySpark RDD Processing

Raw interaction records can be converted into RDDs such as (userID, itemID, rating) and item metadata RDDs such as (itemID, description). map() can transform records, filter() can remove invalid interactions, groupByKey() or reduceByKey() can aggregate ratings, and join() can combine user preferences with item features. Partitioning allows the computation to be distributed across worker nodes. Persisting frequently reused RDDs can reduce repeated computation during recommendation generation.

1.5 Intelligent Agent Interaction

Each recommendation agent observes the current user state, including recent clicks, ratings, search terms and purchase behaviour. The agent maintains a preference profile and decides how strongly content-based and collaborative scores should influence the final ranking. For example, if a user repeatedly selects items belonging to one category, the agent can temporarily increase the weight of content similarity. If the user begins behaving like another group of users, collaborative information can receive greater weight. Feedback is treated as evidence for updating the agent's internal state, allowing recommendations to adapt over time.

1.6 Working Procedure

Collect and clean user, item and interaction data.

Create distributed RDDs and partition the data.

Generate item/content features and user preference profiles.

Calculate content-based similarity scores.

Calculate collaborative filtering scores from interaction patterns.

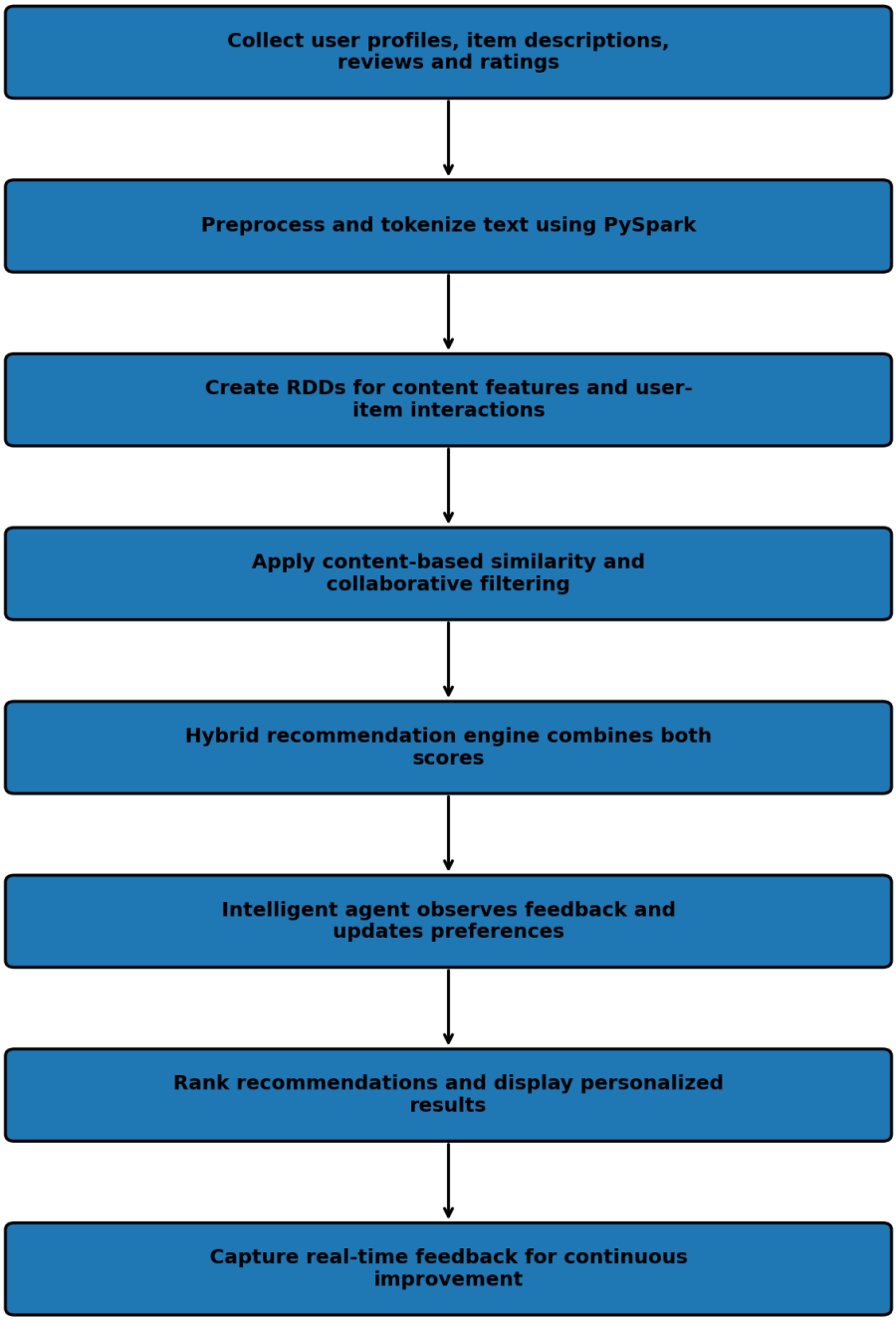
Combine the two scores using a hybrid weighting strategy.

Use an intelligent agent to personalize and re-rank the results.

Present recommendations and capture user feedback.

Update the user model and repeat the process for future interactions.

System Flowchart – Design Task 1



1.7 Advantages

Higher personalization than using a single recommendation method.

Better scalability for large datasets through distributed processing.

Adaptive behaviour based on real-time user feedback.

Can reduce cold-start effects by using item content when interaction history is limited.

Suitable for e-commerce, entertainment, education and media platforms.

1.8 Conclusion

The proposed hybrid recommendation engine combines the strengths of content-based and collaborative filtering while using PySpark RDDs for scalable data processing. Intelligent agents make the system adaptive by observing behaviour and continuously refining user preferences. The resulting architecture can deliver personalized recommendations efficiently even when the number of users and items becomes very large.

2. Smart Disaster Detection System

Problem Statement: Develop an AI-based disaster detection system using PySpark and RDD operations to detect natural disasters such as floods and earthquakes from multiple data streams, including satellite images, sensors and social media. The system should process large datasets efficiently and support intelligent agents in identifying severe events and recommending response strategies.

2.1 Abstract

The Smart Disaster Detection System is a distributed AI platform designed for early detection and monitoring of natural disasters. Modern disasters generate large quantities of heterogeneous data from satellite observations, weather stations, IoT sensors, seismic instruments and social-media reports. Processing these streams centrally can create delays, especially when the affected region is large. PySpark RDDs provide distributed processing for cleaning, transforming and aggregating observations across multiple computing nodes. AI detection modules identify patterns associated with floods, earthquakes and other hazards. Intelligent agents evaluate event severity, compare evidence from multiple sources and coordinate alerts. The system can prioritize affected locations, estimate risk levels and support emergency-response teams with timely information.

2.2 Objectives

Integrate heterogeneous disaster-related data sources.

Process large-scale observations using distributed RDD operations.

Detect abnormal patterns associated with floods, earthquakes and related hazards.

Reduce false alarms by combining evidence from multiple sources.

Provide severity estimates and location-based alerts.

Enable intelligent agents to coordinate monitoring and response decisions.

2.3 System Architecture

Component	Description
Data Sources	Satellite images, rainfall measurements, river levels, seismic sensors, weather stations and social-media reports.
RDD Ingestion Layer	Converts raw observations into distributed records for parallel processing.
Preprocessing	Removes duplicates, handles missing values, normalizes sensor readings and filters irrelevant reports.
Feature Extraction	Derives rainfall intensity, water-level changes, seismic magnitude, location and text-based indicators.
AI Detection Layer	Classifies observations and detects abnormal or disaster-related patterns.
Agent Coordination Layer	Agents compare evidence, estimate severity and prioritize affected regions.
Alert and Response Layer	Generates warnings, maps, priority levels and response recommendations.

2.4 PySpark RDD Operations

RDDs are useful because disaster data can be divided into partitions and processed in parallel. map() can convert sensor records into standardized structures. filter() can remove invalid readings. flatMap() can extract multiple keywords or features from social-media messages. reduceByKey() can calculate regional statistics such as total rainfall or average sensor values. groupByKey() can organize observations by geographical region, while join() can combine sensor evidence with location metadata and historical disaster records.

2.5 Intelligent-Agent Coordination

Multiple intelligent agents can specialize in different evidence sources. A sensor agent monitors numerical measurements, a satellite agent analyzes remote-sensing indicators, a social-media agent extracts public reports, and a decision agent combines the evidence. When several agents independently report abnormal conditions in the same region, the decision agent increases confidence in the event. Agents can also prioritize locations according to population, severity, infrastructure vulnerability and available emergency resources.

2.6 Working Procedure

Collect observations from satellite, sensors, weather systems and social media.

Convert incoming records into distributed RDDs.

Clean and standardize the observations.

Extract numerical, spatial and textual disaster indicators.

Run distributed detection and classification operations.

Compare results from multiple data sources.

Estimate event probability, severity and affected region.

Generate alerts and response priorities.

Continue monitoring and update decisions as new data arrive.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined

Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation